
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский физико-технический институт
(национальный исследовательский университет)»
Физтех-школа Прикладной Математики и Информатики
Кафедра интеллектуальных систем

Направление подготовки / специальность: 01.03.02 Прикладная математика и информатика

Направленность (профиль) подготовки: Прикладная математика и компьютерные науки

СТРУКТУРНОЕ ПРОРЕЖИВАНИЕ МОДЕЛЕЙ ГЛУБОКОГО ОБУЧЕНИЯ НА ОСНОВЕ СУРРОГАТНЫХ МОДЕЛЕЙ

(бакалаврская работа)

Студент:

Иванов Максим Олегович

(подпись студента)

Научный руководитель:

Бахтеев Олег Юрьевич,
канд. физ.-мат. наук

(подпись научного руководителя)

Консультант (при наличии):

(подпись консультанта)

Москва 2026

Abstract

The paper investigates the problem of structural pruning in neural networks. Structural pruning is the procedure of removing entire groups of parameters from a neural network, for example, filters in CNNs or skip connections. An appropriate pruning strategy can improve both generalization performance and inference efficiency. The main difficulty of structural pruning is that removing a layer requires removing or consistently modifying all dependent layers.

The proposed method is based on the analysis of the deep learning computation graph and the estimation of the information flow propagated through it. The method enables the estimation of the importance of operations in the computation graph in a few-shot setting, i.e., using several forward passes of a data subset through the analyzed model. To demonstrate the effectiveness of the proposed method, we conduct multiple experiments on the MNIST dataset.

Contents

Introduction	4
1 Problem statement	7
2 Structural pruning using computation graph information	9
3 Computational experiment	12
3.1 Settings	12
3.2 Experiment results	14
Conclusion	18
Acknowledgements	19
References	20

Introduction

Training neural networks requires increasingly large amounts of computational resources [1]. Developing and training models with novel architectures can be particularly challenging under limited computational budgets [2]. In addition, edge computing applications require neural network inference to be performed on resource-constrained portable devices [3]. Consequently, there is a growing need to improve both the generalization performance and the inference efficiency of neural networks. One of the approaches to addressing the issues mentioned above is model pruning.

By pruning, we mean the task of reducing the size of a network by removing parameters [4]. Pruning is generally classified into unstructured [5, 6] and structured [7, 8] approaches. In the first case, individual irrelevant weights are set to zero, whereas in the second case, entire groups of parameters are removed from the neural network. Several methods for unstructured pruning have been proposed [9, 10, 11, 12]. Most of them rely on removing weights according to an importance criterion, for example, weight magnitude [12, 13], norm-based criteria [14], or estimated loss change [10, 15].

Although unstructured pruning helps reduce the computational cost of inference [12], pruning models with novel architectures remains challenging, especially when dependent layers must be removed simultaneously. This issue is investigated in [16], where structural pruning is performed by constructing a dependency graph to explicitly model inter-layer dependencies and to group coupled parameters for joint pruning. Other works review the structural pruning problem and propose methods that either do not use dataset information [17, 18] or rely only on a small number of samples [17, 19]. The goal is to perform pruning in a zero-shot setting, i.e., without fine-tuning [20, 21]. However, some existing methods are designed for specific architectures, which poses a significant obstacle to applying such algorithms across diverse problems [22, 23].

In this paper, we investigate the problem of structural pruning. We represent the computation graph as a directed graph whose vertices correspond to network layers (see Fig. 1). The proposed method is based on analyzing the deep learning computation graph and estimating the information flow propagated through it. Our goal is to perform structural pruning for arbitrary neural network architectures without restricting the method to a specific family of models. The method enables the estimation of the importance of operations in the computation graph in a few-shot

setting, i.e., using only a small subset of data passed through the analyzed model.

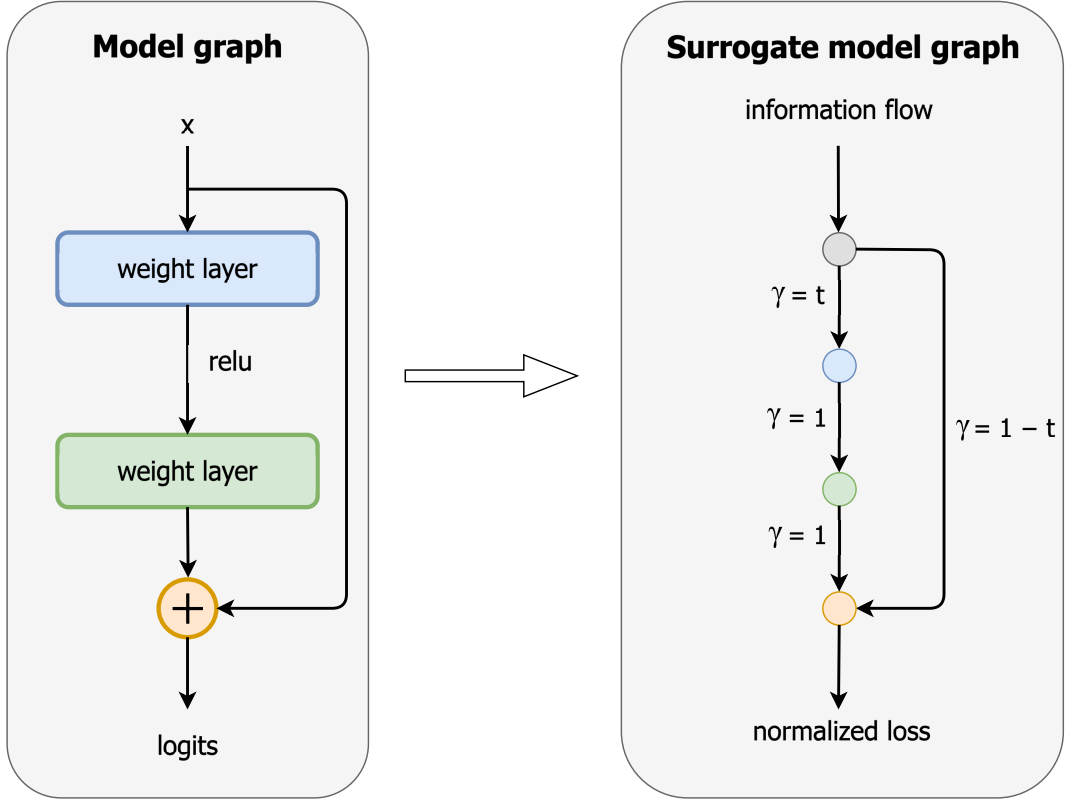


Figure 1: A schematic illustration of a residual connection and the corresponding surrogate model graph. A coefficient γ_{ij} is assigned to each edge; the collection of these coefficients determines the optimized graph structure.

The computational experiments are conducted on the MNIST dataset [24]. The proposed method outperforms surrogate-based baselines that do not replicate the structure of the original model, achieving $1.5\times$ higher correlation metrics.

This improvement is further illustrated using correlation coefficients and scatter plots.

Our main contributions are:

1. We propose a surrogate-based structural pruning method that explicitly incorporates information from the neural network computation graph. The method represents the model as a directed computational graph and introduces structural constraints on the surrogate model derived from the information flow within this graph.
2. We empirically demonstrate that the proposed graph-based constraints improve surrogate reliability in the few-shot setting, enabling more stable and interpretable estimation of operation importance for structural pruning.

3. We validate the proposed approach on multiple models trained on the MNIST dataset [24]. To assess the generality of the method, we consider randomly generated computational graphs inspired by neural architecture search [25] search spaces and show that incorporating computation graph information leads to improved performance compared to surrogate models that ignore the original model structure.

1. Problem statement

We address the problem of structural pruning in deep neural networks formulated in terms of their computation graphs. We define the **computation graph** of a neural network as a directed graph $G_{\text{computation}} = (V, E)$, where each vertex $v_i \in V$ corresponds to a layer (or computational module) of the network, and each directed edge $e_{ij} \in E$ represents the data flow from the output of layer v_i to the input of layer v_j . Formally,

$$e_{ij} \in E \quad \Leftrightarrow \quad \mathbf{h}_j = \mathbf{f}_j(\mathbf{h}_i, \mathbf{w}_j), \quad (1)$$

where $\mathbf{h}_i$ denotes the activation produced by vertex v_i , and $\mathbf{f}_j$ is the transformation implemented at vertex v_j . In this formulation, removing a vertex corresponds to eliminating the entire layer from the network, whereas removing an edge corresponds to removing a data dependency between two layers (e.g., in residual or multi-branch architectures).

We consider the scenario of **edge removal** — deleting selected connections $e_{ij} \in E$ in order to reduce memory consumption and computational cost, possibly at the expense of predictive accuracy. Alternatively, one may prune either vertices, which correspond to layers in the source model, respectively. These pruning strategies can be reduced to one another.

Let the dataset be denoted by $\mathfrak{D} = (\mathbf{X}, \mathbf{y})$, where $\mathbf{X} \in \mathbb{R}^{m \times n}$ is the matrix of input features and $\mathbf{y} \in \mathbb{R}^m$ is the vector of target variables. Let $\mathbf{w} \in \mathbb{R}^k$ be the vector of model parameters. The model is trained by minimizing the error function

$$S = S(\mathbf{w} \mid \mathfrak{D}, f). \quad (2)$$

After pruning, we obtain a modified parameter vector $\mathbf{w}' \in \mathbb{R}^k$, in which certain groups of parameters are set to zero, corresponding to removed edges or vertices in the computation graph. The challenge of structural pruning lies in estimating the importance of parameter groups in terms of their contribution to the network information flow and the final predictive performance.

The following baselines can be found in the literature:

1. **Loss-based pruning** — directly measuring the post-pruning error difference

and selecting $\mathbf{w}'$ to minimize the performance drop:

$$\min_{\mathbf{w}' \in \mathbb{R}^k} |S(\mathbf{w}' \mid \mathfrak{D}, f) - S(\mathbf{w} \mid \mathfrak{D}, f)|. \quad (3)$$

This objective leads to a combinatorial optimization problem and is therefore computationally intractable in practice [26].

2. **Magnitude-based pruning** — removing parameters with the smallest magnitudes. This approach is based on the assumption that the magnitude of a parameter correlates with its importance [12]. In practice, the L_1 -norm is often used to estimate parameter importance.
3. **Taylor-based pruning** — estimating the impact of pruning using a second-order Taylor expansion of the error function around $\mathbf{w}$:

$$S(\mathbf{w}') \approx S(\mathbf{w}) + \langle \nabla S(\mathbf{w}), \Delta \mathbf{w} \rangle + \frac{1}{2} \Delta \mathbf{w}^\top \mathbf{H} \Delta \mathbf{w}, \quad (4)$$

where $\nabla S(\mathbf{w})$ is the gradient of the error function and $\mathbf{H} = \nabla^2 S(\mathbf{w})$ is the corresponding Hessian matrix [26].

The use of local pruning methods often leads to a degradation in performance when multiple layers are pruned simultaneously [7]. Therefore, **Magnitude-based pruning** and **Taylor-based pruning** are not well suited for large-scale structural pruning.

The problem is therefore to design an importance estimation strategy for parameter groups that enables effective structural pruning, i.e., maximizes the reduction in model size and computational complexity while keeping the increase in the error function within acceptable bounds.

2. Structural pruning using computation graph information

Let the dataset be denoted by

$$\mathfrak{D} = \mathfrak{D}_{\text{train}} \cup \mathfrak{D}_{\text{val}}. \quad (5)$$

We employ the loss-based pruning method defined by the optimization problem (3).

The proposed pruning algorithm constructs a **surrogate graph model** $G(\gamma, \mathbf{M})$ that replicates the structure of the original neural network. The graph is derived from the computation graph of the model. For each edge $e_{ij} \in E$, introduce a continuous importance coefficient $\gamma_{ij} \in [0,1]$. Additionally, define a binary edge mask $\mathbf{M} \in \mathbb{Z}_2^{|E|}$ indicating whether an edge is pruned (see Fig. 2). Formally,

$$\mathbf{M}_{ij} = 0 \quad \Leftrightarrow \quad \gamma_{ij} = 0. \quad (6)$$

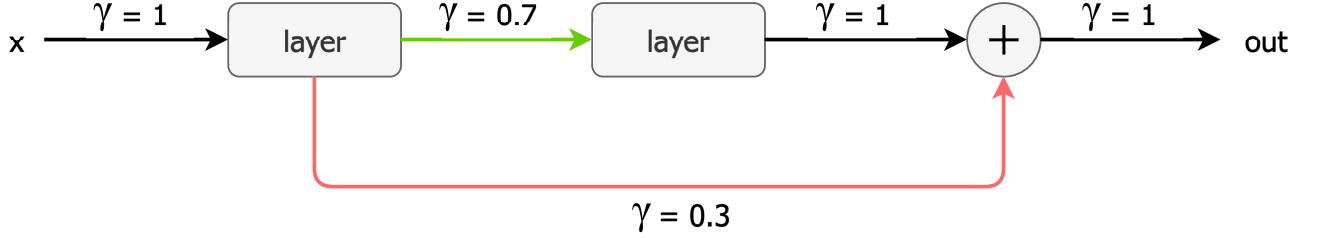


Figure 2: Simplified schematic of a residual connection. A coefficient γ_{ij} is assigned to each edge; smaller values of γ_{ij} indicate lower edge importance.

Lemma 1. *The graph model $G(\gamma, \mathbf{M})$ can be represented as*

$$G(\gamma, \mathbf{M})[\mathbf{x}] = \mathbf{x} \cdot \sum_{p \in P} \prod_{(i,j) \in p} (\gamma_{ij} \cdot \mathbf{M}_{ij}), \quad (7)$$

where P — the set of all paths in the graph.

Proof. Consider an arbitrary path $p \in P$. During the forward pass, the input $\mathbf{x}$ is multiplied by the edge weights γ_{ij} and the corresponding masks $\mathbf{M}_{ij}$ along the path p :

$$\mathbf{x} \cdot \prod_{(i,j) \in p} (\gamma_{ij} \cdot \mathbf{M}_{ij}). \quad (8)$$

Summing these contributions over all paths yields the desired expression. $\square$

We prove the following theorem describing the local behavior of the model.

Theorem 1. *Let $\mathbf{M} = \mathbf{1}$ denote the full mask, i.e.,*

$$\forall (i,j) \in E : \mathbf{M}_{ij} = 1. \quad (9)$$

Then, in a neighborhood of the point $\mathbf{M} = \mathbf{1}$, the error function S can be approximated by the Taylor expansion associated with the graph structure:

$$\begin{aligned} S(\mathbf{M}) = & \sum_{(i,j) \in E} \left. \frac{\partial G}{\partial \mathbf{M}_{ij}} \right|_{\mathbf{M}=\mathbf{1}} (\mathbf{M}_{ij} - 1) \\ & + \frac{1}{2} \sum_{(i,j) \in E} \sum_{(k,l) \in E} \left. \frac{\partial^2 G}{\partial \mathbf{M}_{ij} \partial \mathbf{M}_{kl}} \right|_{\mathbf{M}=\mathbf{1}} (\mathbf{M}_{ij} - 1)(\mathbf{M}_{kl} - 1) \\ & + o(\|\mathbf{M} - \mathbf{1}\|^2). \end{aligned} \quad (10)$$

Proof. For simplicity, consider the scalar-output case.

As discussed above, consider the model represented as a directed acyclic graph. Let $\mathbf{w}_p(\mathbf{x})$ represent the input-dependent contribution of the path p to the network's behavior. Then the model output can be written as

$$f(\mathbf{x}, \mathbf{M}) = \sum_{p \in P} \mathbf{w}_p(\mathbf{x}) \prod_{(i,j) \in p} \mathbf{M}_{ij}. \quad (11)$$

Differentiating with respect to the mask variables in a neighborhood of the point $\mathbf{M} = \mathbf{1}$, we obtain

$$\left. \frac{\partial f}{\partial \mathbf{M}_{ij}} \right|_{\mathbf{M}=\mathbf{1}} = \sum_{p \in P, (i,j) \in p} \mathbf{w}_p(\mathbf{x}), \quad (12)$$

$$\left. \frac{\partial^2 f}{\partial \mathbf{M}_{ij} \partial \mathbf{M}_{kl}} \right|_{\mathbf{M}=\mathbf{1}} = \sum_{p \in P, \{(i,j), (k,l)\} \subset p} \mathbf{w}_p(\mathbf{x}). \quad (13)$$

Using Lemma 1, the corresponding derivatives of the graph model are

$$\left. \frac{\partial G}{\partial \mathbf{M}_{ij}} \right|_{\mathbf{M}=\mathbf{1}} = \mathbf{x} \sum_{p \in P, (i,j) \in p} \prod_{(i',j') \in p} \gamma_{i'j'}, \quad (14)$$

$$\left. \frac{\partial^2 G}{\partial \mathbf{M}_{ij} \partial \mathbf{M}_{kl}} \right|_{\mathbf{M}=\mathbf{1}} = \mathbf{x} \sum_{p \in P, \{(i,j),(k,l)\} \subset p} \prod_{(i',j') \in p} \gamma_{i'j'}. \quad (15)$$

Differentiating the error function S twice with respect to the mask variables in a neighborhood of the point $\mathbf{M} = \mathbf{1}$ yields

$$\left. \frac{\partial^2 S}{\partial \mathbf{M}_{ij} \partial \mathbf{M}_{kl}} \right|_{\mathbf{M}=\mathbf{1}} = \frac{\partial \mathcal{L}}{\partial f} \cdot \frac{\partial^2 f}{\partial \mathbf{M}_{ij} \partial \mathbf{M}_{kl}} + \frac{\partial^2 \mathcal{L}}{\partial f^2} \cdot \frac{\partial f}{\partial \mathbf{M}_{ij}} \cdot \frac{\partial f}{\partial \mathbf{M}_{kl}}, \quad (16)$$

where $\mathcal{L}$ is a twice differentiable loss function.

Observe that the second derivatives in (13) and (15) share the same path-inclusion condition,

$$\{(i,j),(k,l)\} \subset p. \quad (17)$$

These derivatives are nonzero if and only if there exists at least one path $p \in P$ containing both edges (i,j) and (k,l) . Therefore, the local dependency structure between mask variables is determined solely by the graph topology. Consequently, the error function preserves the local dependency structure induced by the computational graph. $\square$

Proposition 1. *We impose the following normalization constraint on the outgoing edges of each vertex:*

$$\sum_{j: e_{ij} \in E} \gamma_{ij} = 1. \quad (18)$$

The method is based on optimizing this surrogate graph model. The final optimization problem is formulated as

$$\begin{aligned} \gamma = \arg \min_{\gamma \in [0,1]^{|E|}} \quad & \mathbb{E}_{\mathbf{M}} \|G(\gamma, \mathbf{M}) - S(\mathbf{w} \mid \mathfrak{D}, f)\|_2^2 \\ \text{s.t.} \quad & \sum_{j: e_{ij} \in E} \gamma_{ij} = 1. \end{aligned} \quad (19)$$

3. Computational experiment

3.1. Settings

This section focuses on the choice of neural network architecture. To conduct the experiments, a handwritten digit classification task based on the MNIST dataset [24] was considered.

To solve this task, we implemented an algorithm for generating a graph neural network architecture. The procedure follows an approach similar to neural architecture search [25]. Let `MLPblock` denote a node consisting of a linear layer followed by the ReLU activation function. Formally,

$$\text{MLPblock}(\mathbf{x}) = \text{ReLU}(\text{Linear}(\mathbf{x})). \quad (20)$$

Define `node_list` as the list of constructed nodes. Initially, `node_list` contains two nodes: the input node and a node that maps the input to a latent representation. Then, `n_iterations` iterations of the algorithm are performed. Each iteration consists of four steps.

Step 1. Independently select two parent nodes from `node_list`.

Step 2. Apply a `MLPblock` to the first parent node selected in Step 1 and append the resulting node to `node_list`.

Step 3. Apply a `MLPblock` to the second parent node selected in Step 1 and append the resulting node to `node_list`.

Step 4. Apply a `MLPblock` to combine the outputs obtained in Steps 2 and 3, thereby creating a new node, and append it to `node_list`.

After completing all iterations, the algorithm aggregates all terminal (hanging) nodes and applies a final classifier for the MNIST task.

Depending on the value of `n_iterations`, the resulting architecture contains a different number of edges (see Figs. 3, 4).

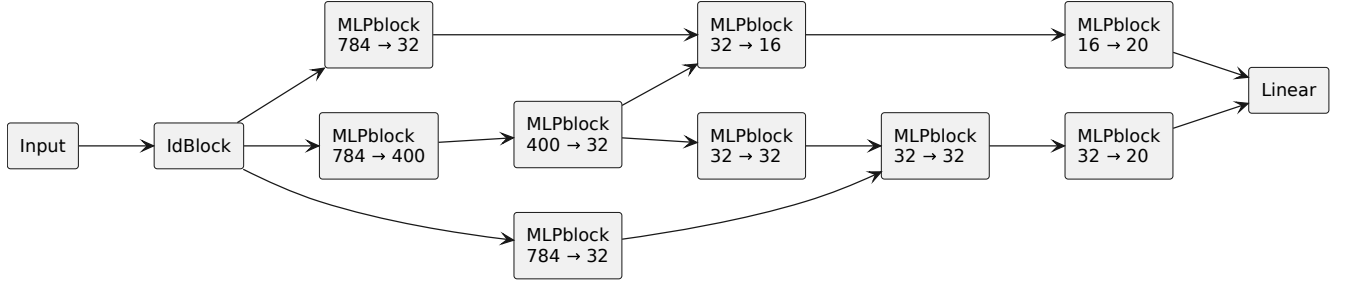


Figure 3: The generated graph with 14 edges.

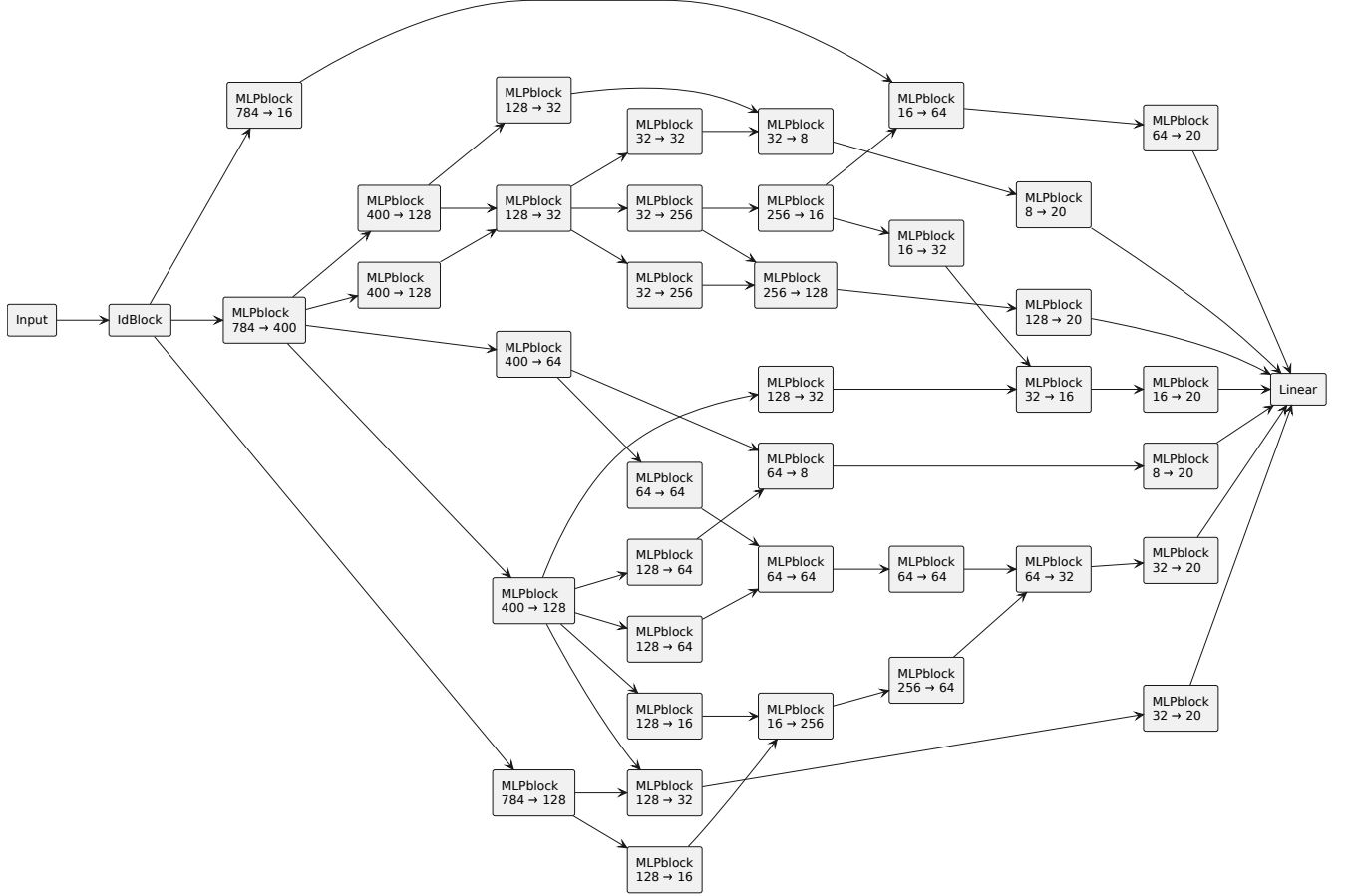


Figure 4: The generated graph with 56 edges.

The training set for the surrogate models consists of binary masks containing either all ones, all zeros, or exactly one zero with all remaining entries equal to one. Thus, the size of the training set is equal to (number of edges + 1), which enables a few-shot setting. The test set consists of 500 masks sampled uniformly without replacement.

The following models were compared in the experiments:

- **Graph:** The proposed surrogate graph model.

- **FCN**: A fully connected network with the following architecture:
Linear(number_of_edges, 10), ReLU(), Linear(10, 1).
- **Linear**: Linear regression.
- **L1**: Magnitude pruning [12].
- **Taylor expansion**: The method using the first and second-order Taylor expansions [26].

The **surrogate graph model** is trained for 3000 epochs, while the **FCN model** is trained for 1000 epochs. Both models are optimized using the mean squared error (MSE) loss and the Adam optimizer with learning rate 10^{-3} .

In the experiments, Spearman and Kendall correlation coefficients between the true and predicted error values were used as evaluation metrics. These statistics assess the rank dependence between samples. For a more robust evaluation, each experiment was repeated ten times, and the results were averaged.

3.2. Experiment results

As shown in Tables 1 and 2, the proposed method demonstrates strong performance in the few-shot setting. It significantly outperforms the baseline models and provides informative estimates of edge importance.

Table 1: Average Spearman correlation coefficient over 10 runs.

Model	10–15 edges	20–40 edges	50–100 edges	> 100 edges
Graph	0.299 ± 0.235	0.296 ± 0.178	0.349 ± 0.128	0.411 ± 0.057
FCN	-0.009 ± 0.213	-0.060 ± 0.187	-0.028 ± 0.090	-0.053 ± 0.099
Linear	0.060 ± 0.187	0.201 ± 0.122	0.248 ± 0.090	0.294 ± 0.047
L1	0.547 ± 0.063	0.505 ± 0.063	0.369 ± 0.060	0.280 ± 0.081
Taylor exp.	0.119 ± 0.247	-0.052 ± 0.082	-0.136 ± 0.076	-0.204 ± 0.044

Table 2: Average Kendall correlation coefficient over 10 runs.

Model	10–15 edges	20–40 edges	50–100 edges	> 100 edges
Graph	0.253 ± 0.202	0.242 ± 0.146	0.282 ± 0.105	0.329 ± 0.045
FCN	-0.011 ± 0.151	-0.042 ± 0.128	-0.020 ± 0.061	-0.036 ± 0.066
Linear	0.039 ± 0.135	0.137 ± 0.083	0.167 ± 0.061	0.199 ± 0.032
L1	0.401 ± 0.057	0.353 ± 0.049	0.259 ± 0.041	0.192 ± 0.057
Taylor exp.	0.094 ± 0.182	-0.033 ± 0.055	-0.090 ± 0.051	-0.136 ± 0.030

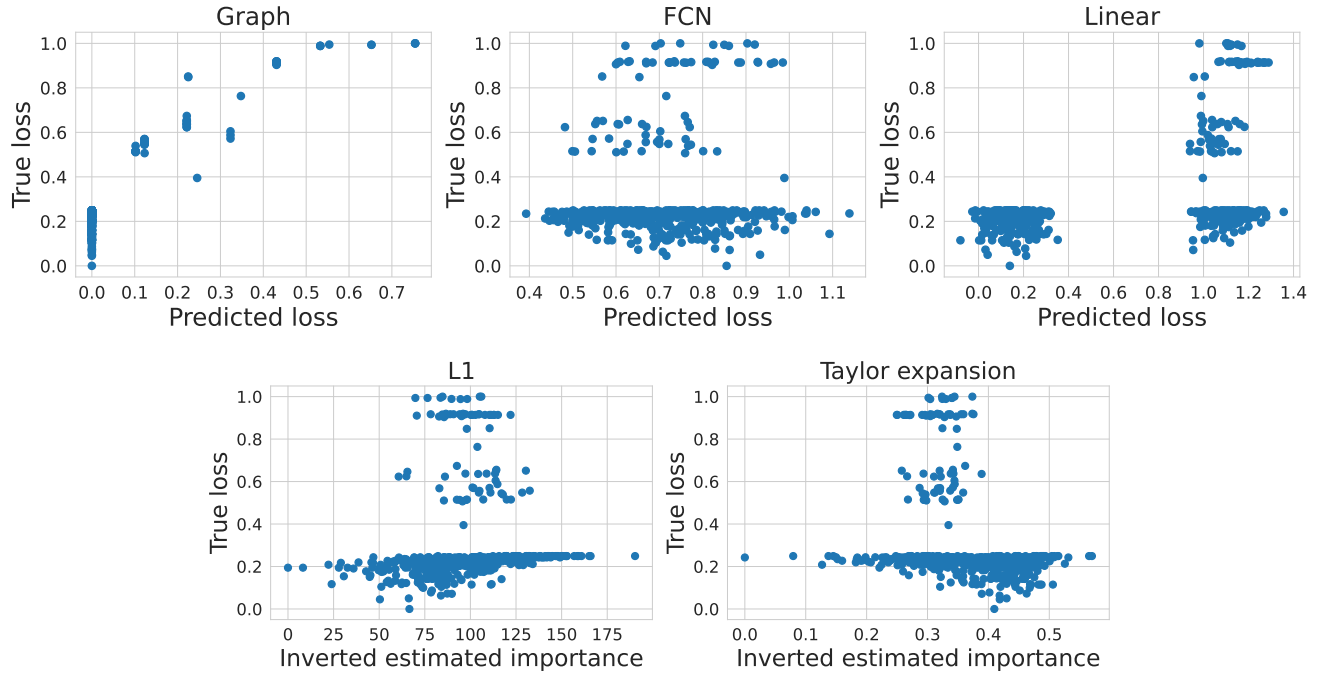


Figure 5: Scatter plot of true and predicted error values for the baseline comparison. The parameter `n_iterations` is set to 2.

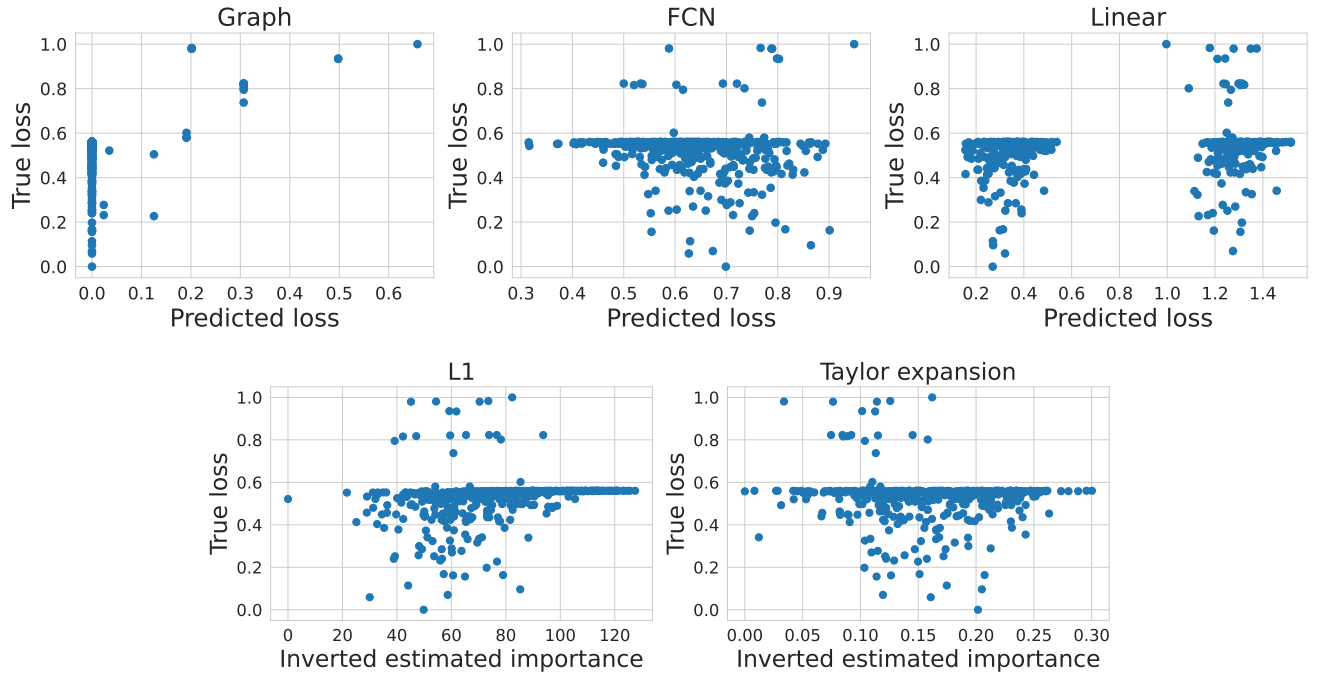


Figure 6: Scatter plot of true and predicted error values for the baseline comparison. The parameter `n_iterations` is set to 5.

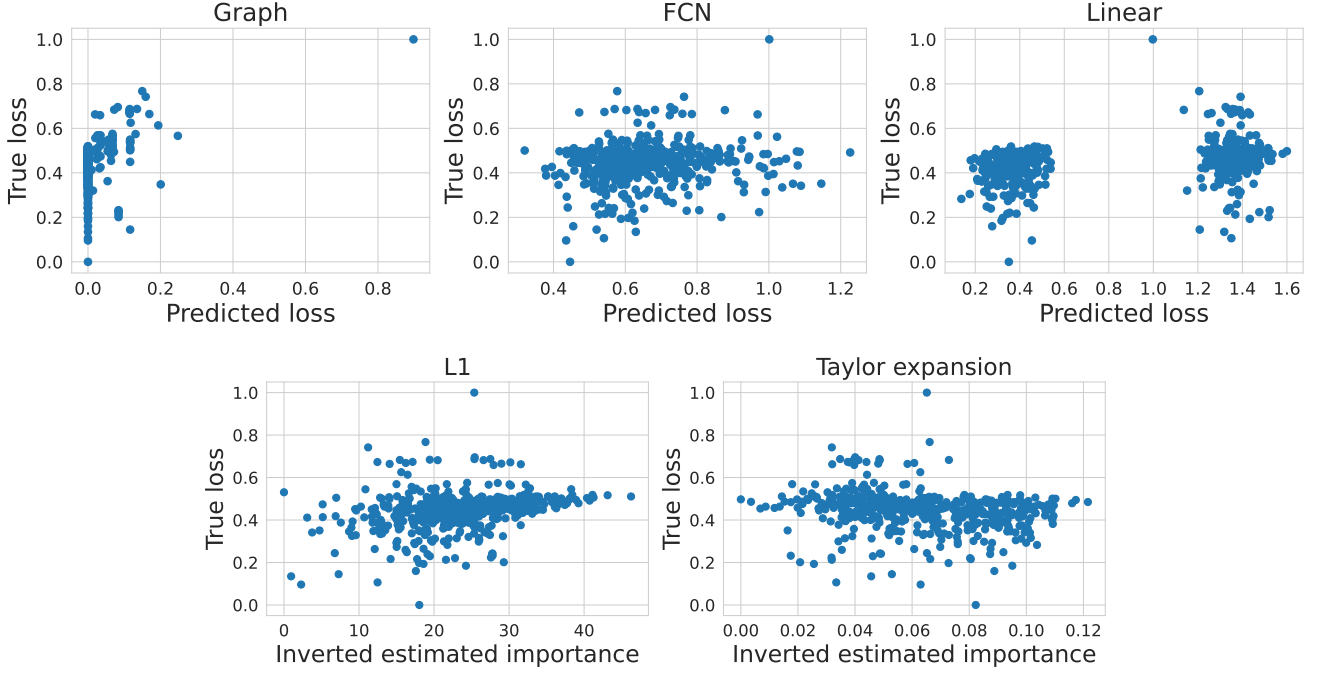


Figure 7: Scatter plot of true and predicted error values for the baseline comparison. The parameter `n_iterations` is set to 14.

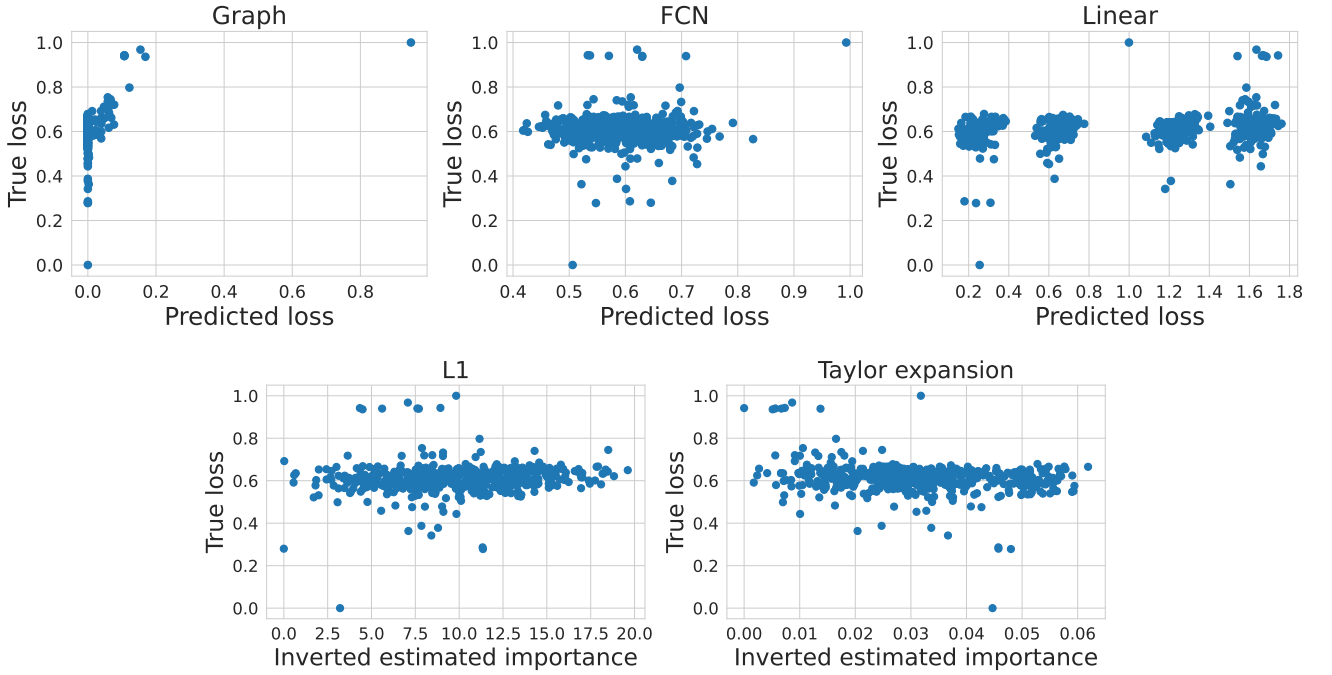


Figure 8: Scatter plot of true and predicted error values for the baseline comparison. The parameter `n_iterations` is set to 30.

Additionally, scatter plots (Figs. 5–8) illustrate the relationship between true and predicted error values. In all considered cases, the surrogate graph model exhibits a stronger dependence between true and predicted errors than the baseline models. These visualizations are consistent with the correlation metrics reported above.

We compute the overlap between the top- $k\%$ of models ranked according to the true loss and the top- $k\%$ of models ranked according to the surrogate prediction. The resulting overlap ratio is used as a measure of prediction accuracy. To improve statistical stability, the experiment is repeated 10 times and the results are averaged. The results are presented in Tables 3 and 4.

Table 3: The overlap between the top-5% of models ranked according to the true loss and the models ranked according to the surrogate prediction.

Model	10–15 edges	20–40 edges	50–100 edges	more than 100 edges
Graph	62.0%	64.5%	69.3%	58.3%
FCN	2%	15.2%	5.2%	6.8%
Linear	18.8%	9.6%	27.2%	15.2%
L1	0.0%	1.6%	6.8%	6.8%
Taylor expansion	8.0%	10.8%	29.6%	24.4%

Table 4: The overlap between the top-10% of models ranked according to the true loss and the models ranked according to the surrogate prediction.

Model	10–15 edges	20–40 edges	50–100 edges	more than 100 edges
Graph	55.3%	54.8%	67.4%	66.9%
FCN	7.4%	16.6%	12.4%	11.4%
Linear	22.6%	15.0%	28.6%	25.8%
L1	2.0%	4.0%	12.0%	10.4%
Taylor expansion	12.6%	16.8%	30.0%	29.6%

The plots in the (Fig. 9) indicate that, as the number of edges in the graph model increases, the proposed method achieves higher performance.

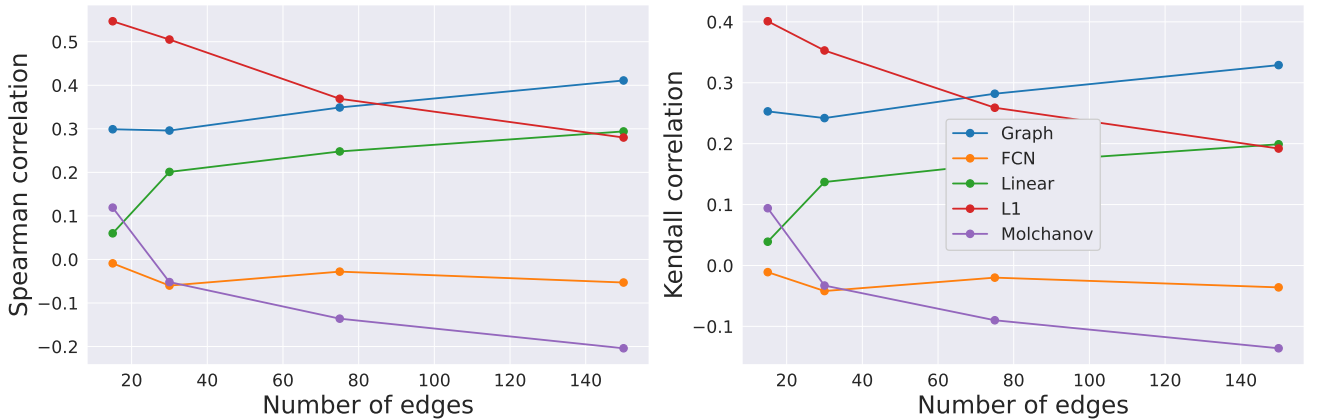


Figure 9: The plot shows the dependence of correlation coefficients on the number of edges.

Conclusion

This paper investigated the problem of structural pruning in neural networks. The proposed method is based on analyzing the deep learning computation graph and estimating the information flow propagated through it. The method enables the estimation of the importance of operations in a computation graph in a few-shot setting. Experiments have shown that the method is superior to surrogate models that do not use computation graph information. It is also possible to fine-tune the model after pruning, which can significantly improve its performance. Thus, the important role of computation graph information in structural pruning has been demonstrated. In future work, we plan to extend and generalize the method to channel-wise pruning (for example, filters in CNNs).

Acknowledgements

The authors would like to thank Andrey Ryzhov for providing the computational resources used to conduct the experimental evaluations in this paper.

References

1. Compute Trends Across Three Eras of Machine Learning / Jaime Sevilla, Lennart Heim, Anson Ho et al. // *2022 International Joint Conference on Neural Networks (IJCNN), Padua, Italy, 2022, pp. 1-8.* — 2022. — июль. — Pp. 1–8.
2. *Tan Mingxing, Le Quoc V.* EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks // *International Conference on Machine Learning, 2019.* — 2019. — май.
3. *Banjanović-Mehmedović Lejla, Husaković Anel.* Edge AI: Reshaping the Future of Edge Computing with Artificial Intelligence // BASIC TECHNOLOGIES AND MODELS FOR IMPLEMENTATION OF INDUSTRY 4.0. — Basic 4.0. — Academy of Sciences and Arts of Bosnia and Herzegovina, 2023. — октябрь. — Pp. 133–160.
4. Pruning and Quantization for Deep Neural Network Acceleration: A Survey / Tailin Liang, John Glossner, Lei Wang et al. — 2021. — январь.
5. Group Fisher Pruning for Practical Network Compression / Liyang Liu, Shilong Zhang, Zhanghui Kuang et al. — 2021. — август.
6. *Wang Chaoqi, Zhang Guodong, Grosse Roger.* Picking Winning Tickets Before Training by Preserving Gradient Flow // *In Proceedings of the 8th International Conference on Learning Representations (ICLR), 2020.* — 2020. — февраль.
7. Gate Decorator: Global Filter Pruning Method for Accelerating Deep Convolutional Neural Networks / Zhonghui You, Kun Yan, Jinmian Ye et al. — 2019. — сентябрь.
8. *Ma Xinyin, Fang Gongfan, Wang Xinchao.* LLM-Pruner: On the Structural Pruning of Large Language Models // *Advances in Neural Information Processing Systems* / Ed. by A. Oh, T. Naumann, A. Globerson et al. — Vol. 36. — Curran Associates, Inc., 2023. — Pp. 21702–21720.
9. *Lecun Yann, Denker John, Solla Sara.* Optimal Brain Damage. — Vol. 2. — 1989. — 01. — Pp. 598–605.

10. *Hassibi Babak, Stork David*. Second order derivatives for network pruning: Optimal Brain Surgeon // *Advances in Neural Information Processing Systems* / Ed. by S. Hanson, J. Cowan, C. Giles. — Vol. 5. — Morgan-Kaufmann, 1992.
11. *Zeng Wenyuan, Urtasun Raquel*. MLPrune: Multi-Layer Pruning for Automated Neural Network Compression. — 2019. — URL: <https://openreview.net/forum?id=r1g5b2RcKm>.
12. Learning both Weights and Connections for Efficient Neural Networks / Song Han, Jeff Pool, John Tran, William J. Dally. — 2015. — июнь.
13. *Lubana Ekdeep Singh, Dick Robert P*. A Gradient Flow Framework For Analyzing Network Pruning. — 2020. — сентябрь.
14. Soft Filter Pruning for Accelerating Deep Convolutional Neural Networks / Yang He, Guoliang Kang, Xuanyi Dong et al. — 2018. — август.
15. Pruning Convolutional Neural Networks for Resource Efficient Inference / Pavlo Molchanov, Stephen Tyree, Tero Karras et al. — 2016. — ноябрь.
16. DepGraph: Towards Any Structural Pruning / Gongfan Fang, Xinyin Ma, Mingli Song et al. — 2023. — январь.
17. Pruning neural networks without any data by iteratively conserving synaptic flow / Hidenori Tanaka, Daniel Kunin, Daniel L. K. Yamins, Surya Ganguli // *Advances in Neural Information Processing Systems 2020*. — 2020. — июнь.
18. RED : Looking for Redundancies for Data-Free Structured Compression of Deep Neural Networks / Edouard Yvinec, Arnaud Dapogny, Matthieu Cord, Kevin Bailly. — 2021. — май.
19. Towards efficient network compression via Few-Shot Slimming / Junjie He, Yinzhang Ding, Ming Zhang, Dongxiao Li // *Neural Networks*. — 2022. — Vol. 147. — Pp. 113–125. — URL: <https://www.sciencedirect.com/science/article/pii/S0893608021004846>.
20. Only Train Once: A One-Shot Neural Network Training And Pruning Framework / Tianyi Chen, Bo Ji, Tianyu Ding et al. — 2021. — июль.

21. *Liu Dan, Liu Xue*. Pruning On-the-Fly: A Recoverable Pruning Method without Fine-tuning. — 2022. — декабрь.
22. *Frantar Elias, Alistarh Dan*. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. — 2023. — январь.
23. Architecture-aware Network Pruning for Vision Quality Applications / Wei-Ting Wang, Han-Lin Li, Wei-Shiang Lin et al. — 2019. — август.
24. *Deng Li*. The MNIST Database of Handwritten Digit Images for Machine Learning Research [Best of the Web] // *IEEE Signal Processing Magazine*. — 2012. — ноябрь. — Vol. 29, no. 6. — Pp. 141–142.
25. Learning Transferable Architectures for Scalable Image Recognition / Barret Zoph, Vijay Vasudevan, Jonathon Shlens, Quoc V. Le. — 2017. — июль.
26. Importance Estimation for Neural Network Pruning / Pavlo Molchanov, Arun Mallya, Stephen Tyree et al. — 2019. — июнь.